

④ The Most Important React Hooks - Content ④

Date: / / Page no:

- ① useState
- ② useReducer
- ③ useEffect
- ④ useRef
- ⑤ Props Drilling with real-life example
- ⑥ useContext
- ⑦ useLayoutEffect
- ⑧ Problem without useCallback
- ⑨ useCallback
- ⑩ useMemo → with [useCallback is useMemo]
- ⑪ custom hook → { for Data Fetching :- (useFetch) }

HAPPY - HACKING - & LEARNING.

④ The Most Important React Hooks ✖

Date: / / Page no:

① ④ useState :-

The 'useState' hook is a hook, that is part of the Hooks API introduced in React 16.8.

'useState' is used to declare state variable in functional components, enabling them to hold & update state over the time.

App.jsx :- import { useState } from 'react';

```
const APP = () => {
```

```
  const [counter, setCounter] = useState(0);
```

↓ ↓ ↓
variable name function Initial Value

```
  return (
```

```
    <>
```

```
    <h1> {counter} </h1>
```

```
    <button onClick = {() => setCounter(counter+1)} > Increase </button>
```

```
    <button onClick = {() => setCounter(counter-1)} > Decrease </button>
```

```
    <>
```

```
  )
```

```
  }
```

```
  export default App;
```


② ④ useReducer :- The 'useReducer' hook is another state management hook in React. It is particularly useful when you have complex state logic that involves multiple sub-values or when the next state depends on the previous one. It's an alternative to 'useState' & is often preferable when dealing with more intricate state management.

④ App.jsx :- import { useReducer } from 'react';

// Reducer function - takes the current state & action, return a brand new state

```
const counterReducer = (state, action) => {
  switch (action.type) {
    case 'INCREMENT':
      return { count: state.count + 1 };
    case 'DECREMENT':
      return { count: state.count - 1 };
    default:
      return state;
  }
};
```

```
const App = () => {
```

// useReducer returns the current state & a dispatch function

```
const [state, dispatch] = useReducer(counterReducer, {count: 0});
```

↓ ↓
Reduce function Initial state

```
return ( <
```

```
<button onClick={() => dispatch({type: 'INCREMENT'})}>
```

Increment </button>

```
<button onClick={() => dispatch({type: 'DECREMENT'})}>
```

Decrement </button>

```
</> );
```

export default App;

③ useEffect :-

The 'useEffect' hook is used in React functional components to perform side effects in your components. Side effects can include things like data fetching, subscriptions, manually changing the DOM, 'useEffect' is called after the component renders & each update.

④ App.jsx :- import { useState, useEffect } from 'react';

```
const App = () => {
```

```
  const [data, setData] = useState([]);
```

// useEffect to fetch data from JSONPlaceholder API (Any API)

```
  useEffect(() => {
```

```
    const fetchData = async () => {
```

```
      const api = await fetch("https://jsonplaceholder.typicode.com/posts");
```

```
      const result = await api.json();
```

```
      console.log(result);
```

```
      setData(result); // updating the state with fetched data
```

```
    }(), // Calling the fetchData function
```

```
  }, []); // The empty array [] (dependency array) means  
          this effect runs once after the initial render
```

④ Note :- Maybe you will get two output in console, when you console logging the data { console.log(result) }, because of Life-cycle methods.

④ useRef:- The 'useRef' hook is a utility hook in React that provides a mutable object called a 'ref' object. This object has a 'current' property that can be used to hold a mutable reference to a value. Unlike state, changes to the 'ref' object do not trigger a re-render of the component.

④ App.jsx:- import React, { useRef } from 'react';

```
const App = () => {
```

```
  // creating a ref object
```

```
  const myRef = useRef(null);
```

```
  // function to focus on the input
```

```
  const focusInput = () => {
```

```
    myRef.current.value = "";
```

```
    myRef.current.focus();
```

```
  return (
```

```
    <> /* Assigning the ref object to a DOM element */
```

```
    <input ref = { myRef } type = "text" />
```

```
    <button onClick = { focusInput } > Focus </button>
```

```
  </>
```

```
);
```

```
}
```

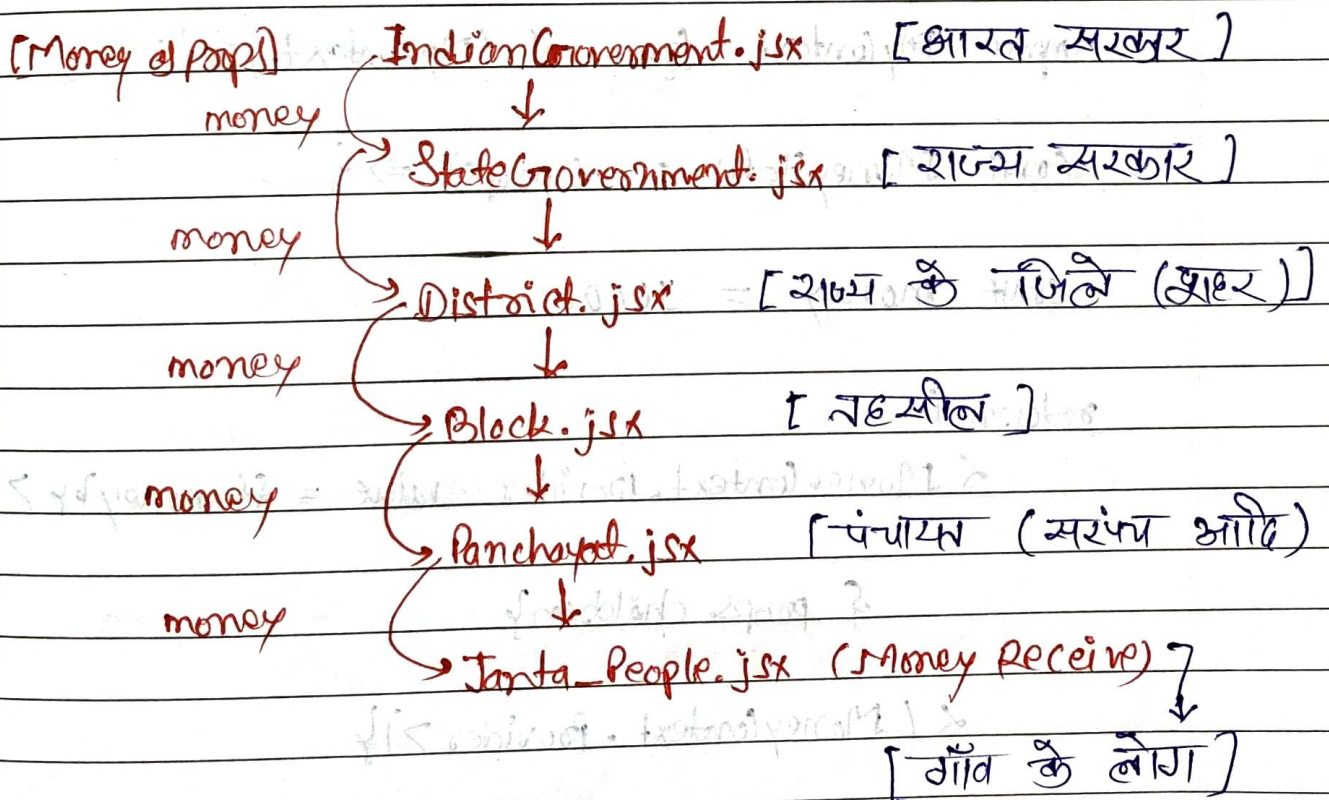
```
export default App;
```


⑤ ④ useContext :-

The 'useContext' hook is a React hook that allows the functional components to consume values from a React context. It provides a way for components to access values from a context without having to explicitly pass them through each level of the component tree as props.

⑥ Props Drilling :-

Suppose Indian Government wants to send some money to his janta (people) then, first he need to send money to "State Government" then "District" then "Blocks" then "Panchayat" then "Janta (people)".



भारत सरकार को अपने जनता को धन राशि भेजने के लिए इन सबसे गुजरना पड़ता था, जहाँ वह money useContext नहीं आया था।

④ Let's understand useContext :-

① Step ① :- Create a Context

// MyContext.jsx :-

```
import { createContext } from 'react';
```

```
const MyContext = createContext();
```

```
export default MyContext;
```

② Step ② :- Create a provider Component

④ MoneyState.jsx :-

```
import MyContext from './MyContext.jsx';
```

```
const MoneyState = (props) => {
```

```
  const money = 10000;
```

```
  return (
```

```
    < MyContext.Provider value = {{ money }} >
```

```
      { props.children }
```

```
    < / MyContext.Provider > )
```

```
  export default MoneyState;
```


(iii) Step ③:- Wrap index.jsx or App.jsx with state [MoneyState]

④ App.jsx:- import React, from 'react';
import people from './People.jsx';
import MoneyState from './MoneyState';

const App = () => {

return (<

<MoneyState>

< People /

</MoneyState> </>)

export default App;

(iv) Step ④:- use the context (MoneyContext) in child component using useContext hook.

④ People.jsx:-

import { useContext } from 'react';

import MoneyContext from './MoneyContext';

const People = () => {

const data = useContext(MoneyContext)

console.log(data)

return (<

<h2> Money = { data.money } </h2>

</>

export default People;

⑥ useLayoutEffect :-

useEffect & useLayoutEffect both hooks are used for performing side effects. useLayoutEffect runs before useEffect.

⑦ When we use useLayoutEffect :-

⑧ Synchronizing State with Layout Changes :-

If you need to make sure that certain state updates are reflected in the DOM before the browser paints, 'useLayoutEffect' can be more suitable than 'useEffect'.

For example, if you're updating a style based on state you want to make sure the style changes are applied before the browser repaints, might use 'useLayoutEffect'.

⑨ Test.jsx :-

import React, { useState, useLayoutEffect } from 'react'

```
const Test = () => {
  const [bgColor, setBgColor] = useState('yellow');
```

```
  useLayoutEffect(() => {
    document.body.style.backgroundColor = bgColor;
  }, [bgColor]);
```

```
  return (<
```

```
    <button onClick={() => setBgColor('blue')}> changeColor </button>
  </>
  );
};
```

```
export default Test;
```


⑦ useCallback :-

The 'useCallback' hook in React is used to memoize functions, especially when passing functions down to child components. It helps optimize performance by preventing unnecessary re-creation of functions during re-renders.

⑧ Problem without useCallback :-

⑨ ParentComponent.jsx :-

```
import { useState } from 'react';
import ChildComponent from './ChildComponent'
```

```
const ParentComponent = () => {
```

```
  const [counter1, setCounter1] = useState(0);
```

```
  const [counter2, setCounter2] = useState(0);
```

```
  const handleClick = () => setCounter1(counter1 + 1);
```

```
  return ( <
```

```
    <p> Count: { counter2 }
```

// passing 'handleClick' function as a prop to ChildComponent.

```
    <ChildComponent handleClick={handleClick} />
```

```
    <button onClick={() => setCounter2(counter2 + 1)}>
```

```
      Increase </button> 
```

```
  </> ) }
```

```
  export default ParentComponent;
```


④ ChildComponent.jsx :-

```
import { useEffect } from 'react';

const ChildComponent = ({ handleClick }) => {

  useEffect(() => {
    console.log("UseEffect is Running");
  }, [handleClick])

  return (
    <ChildComponent
      </>); }

export default ChildComponent;
```

④ Solution :- Write the function logic inside the usecallback

④ For ParentComponent.jsx :-

```
import { usecallback } from 'react';

const handleClick = usecallback(() => {
  setCounter1(counter1 + 1)}, [counter1])
```

When this thing change, then only
it will create new function

8. ④ useMemo:-

The 'useMemo' hook is a memoization hook in React that helps optimize performance by the memoizing the result of a function or computation.

It's used to memoize expensive calculations so that they are only recomputed when the dependencies change.

④ Key Difference between useCallback & useMemo :-

- ① useCallback :- memoize the function.
- ② useMemo :- memoize the function return value/result.

④ App.jsx:- import React, {useState, useMemo} from 'react';
 const App = () => {
 const [inputValue, setInputValue] = useState('');

// Expensive Calculation function

const calculateResult = (input) => {
 console.log("Calculating result --");
 return input.length * 100; }

// memoize the result of the calculation based on the inputValue

const memoizedResult = useMemo(() => calculateResult(inputValue),
 [inputValue]);

return (

<> <input type="text" value={inputValue}
 onChange = {e => setInputValue(e.target.value)} />

<p> Result : {memoizedResult} </p>

</> ; }

export default App;

③ Custom Hook :-

Creating a custom hook for data fetching in React involves encapsulating the logic related to fetching data in a reusable function.

Your function name should be start with "use"

④ CustomHook.jsx :-

import { useState, useEffect } from 'react';

// custom hook for data fetching

const useFetch = (url) => {

const [data, setData] = useState([]);

const [loading, setLoading] = useState(true);

const [error, setError] = useState("");

useEffect(() => {

const fetchData = async () => {

try {

const apiCall = await fetch(url);

const result = await apiCall.json();

setData(result);

} catch (error) {

setError(error);

finally {

setLoading(false); }

fetchData(); // calling fetchData function.

}, [url]);

return { data, loading, error }; }

export default useFetch; customHook;

④ How to use Custom Hook (useFetch) :-

④ Test.jsx :- import React from 'react';
import useFetch from './useFetch CustomHook';

const Test = () => {

const { data, loading, error } = useFetch('https://api.example.com');

if (loading) return <p>Loading:- </p>;

if (error) return <p>Error: {error.message}</p>;

return (

<h1> Data </h1>

{ data.map((d) => console.log(d)) }

</>)

export default Test;